

NeurIPS 2023 | 让强化学习像人一样，随手借用、组合、替换技能

UC San Diego 与 UC Santa Barbara 提出 KGRL 范式与 KIAN 架构，把外部知识变成可自由重排模块

一个人学骑摩托车时，往往不是从零开始：他会借用滑板上练出的平衡感、自行车上练出的踩踏与转向，再把这些习惯拼成新的技能。强化学习（Reinforcement Learning, RL）智能体却很难这样学习。哪怕是人类几分钟就能上手任务，智能体也可能需要上百万次与环境的交互，因为它没有一条简便的路径去复用自己或别人已经掌握的技能。

为缩小这个差距，研究者尝试把外部知识策略（knowledge policy）注入 RL：手写规则、预训练控制器，或行为克隆得到的专家，用它们把智能体推向更好的动作。这类方法确实有帮助，但大多把知识固化得很死：融合机制依赖策略的数量和排列顺序，于是新增一个技能、删掉一个无用策略、或者重排整套知识，都要改动网络结构或重新训练大部分模型。知识虽然在，却谈不上模块化。

NeurIPS 2023 上，来自 UC San Diego 与 UC Santa Barbara 的一篇论文正面处理了这种僵化。它提出知识落地强化学习（Knowledge-Grounded Reinforcement Learning, KGRL）范式，把多个外部知识策略与智能体自己的可学习策略融合起来；并提出知识融合注意力网络（Knowledge-Inclusive Attention Network, KIAN）这一 actor 架构，让这些策略可以自由重排、增添或替换，而无需重新训练网络其余部分。在此过程中，作者还识别并修复了一个探索层面的隐患，即熵失衡（entropy imbalance），一种会拖累探索效率的现象。

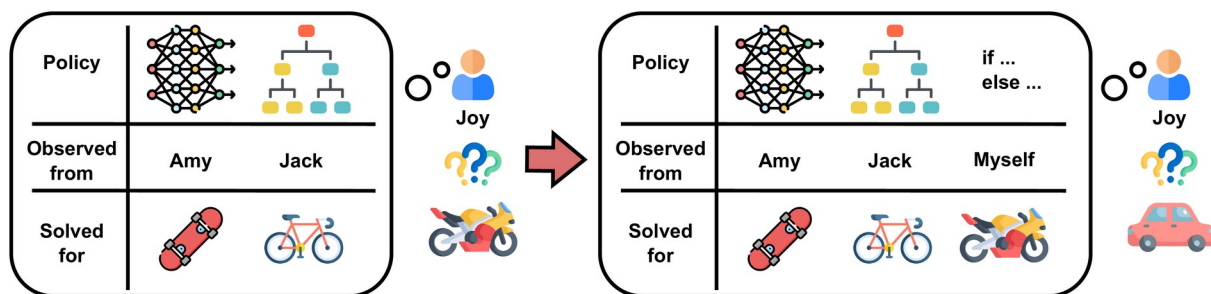


图 1：KGRL 所追求的三种性质：可获取知识、可组合、可增量。Joy 先通过观察 Amy 玩滑板、Jack 骑自行车学会骑摩托车；随后在知识集里加入她自己练出的摩托车策略，再去学开车。技能被观察、复用，并可随时加入。

论文地址：<https://arxiv.org/abs/2210.03729>

代码地址：<https://github.com/Pascalson/KGRL.git>

问题：无法重排的知识

作者把高效、类人的学习归纳为五种性质：可获取知识（能吸纳外部策略）、样本高效、可泛化到新任务、可组合（能把技能拼起来）、可增量（能随时间扩充技能集）。现有的知识引导 RL 方法能满足其中一部分，却在灵活性上受阻。

像 KoGuN、A2T 这类方法，其外部策略融合机制要么绑定固定的输入顺序，要么依赖逐策略的参数。问题的症结正在这种耦合：每个策略的贡献都和其他策略的数量、位置纠缠在一起，因此你无法在不重建融合模块、不重新训练的前提下换掉一个策略或插入一个新策略。这样一来，一个训练好的智能体也就无法把知识干净地迁移到新任务上，恰恰与人复用技能的方式相反。

KIAN：把每个策略编码成注意力的 key

KGRL 保留智能体自身的策略，即内部 actor（inner actor），它始终能从零学出一个解；再把它与一组外部知识策略融合。核心设计问题是：如何组合它们才能让整套知识保持模块化。KIAN 的答案是：不用策略在列表中的位置来描述它，而用一个可学习的嵌入向量，即它的 key（键）来描述它。

KIAN 由三部分组成。内部 actor 是一个普通的可学习策略，因此即便所有外部策略对当前任务都没用，智能体依然能自己找到解。每个知识策略（包括内部 actor）都配有一个可学习的 key，用来表示整个策略本身，与任何具体的状态或动作无关。随后，模型从观测生成一个依赖状态的 query（查询），把它与每个 key 做点积；对这些分数做 softmax 得到注意力权重，最终策略就是所有策略的加权和。

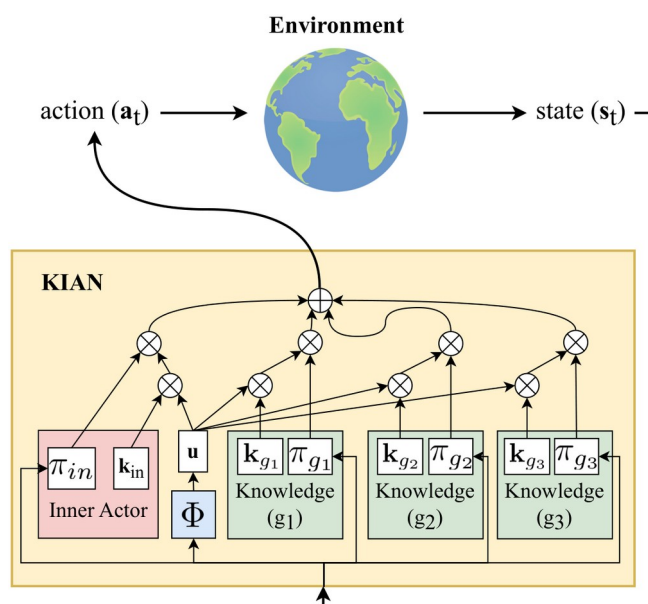


图 2：KIAN 架构。内部 actor 发展智能体自己的策略，外部知识策略 g_1 、 g_2 、 g_3 则通过基于嵌入的注意力式动作预测被融合。每个策略由自己的 key 寻址，因此知识可以自由重排、增添或删除，而无需改动网络其余部分。

具体来说，query $\Phi(s_t)$ 与每个 key（内部 actor 的 k_{in} 、知识策略 j 的 k_{gj} ）做点积、归一化，再经 softmax 得到融合权重；实际执行的策略为 $\pi(\cdot|s_t) = \hat{w}_{in} \cdot \pi_{in}(\cdot|s_t) + \sum_j \hat{w}_{gj} \cdot \pi_{gj}(\cdot|s_t)$ 。因为每个 key 都处在同一个共享嵌入空间中、且能独立成立，这些策略是无序的：加一个策略就是加一个 key，删一个就是删掉它的 key，换一个就是替换它的 key，KIAN 其余部分完全不用改。对连续控制，模型用 Gumbel-softmax 选出一个策略，并用重参数化技巧（reparameterization trick）采样，从而让整个 actor 端到端可微。

修复熵失衡

论文的第二项技术贡献更为微妙。在最大熵 RL（如 SAC 用来鼓励探索的做法）中，智能体会最大化动作分布的熵。当这个分布是多个策略的融合时，作者证明熵项会不均匀地推动各策略的权重，过度偏向某些策略，同时让另一些策略缺乏探索。他们把这一现象称为熵失衡，证明了它出现的条件，并提出修改后的策略分布，重新平衡目标函数，使得在融合大量策略时探索仍然健康。

结果：唯一在所有环境都成功的方法

论文在两套基准上测试：MiniGrid，一组难度递增的离散网格世界任务（Empty-Random、Unlock、DoorKey、Dynamic-Obstacles、LavaCrossing、MultiRoom、KeyCorridor）；以及 OpenAI-Robotics 连续控制的机械臂操作任务（Push、Slide、Pick-and-Place、Reach）。关键在于，所有方法都从同一套初始知识集出发，这套知识由刻意设计得次优的 if-else 程序构成，比如「拿起一把钥匙」或「朝目标移动」这类简单例程，单独任何一个都无法完成任务。所有实验用 10 个随机种子，并报告 95% 置信区间。

在这种次优知识下，KIAN 是唯一在每个环境都成功的方法，也是样本效率最高的方法，而且任务越难，它的优势越大。在 MultiRoom-N4-S5、KeyCorridorS3R2 等最难的网格任务上，各基线（BC、RL、RL+BC、KoGuN、A2T）几乎贴地不动，KIAN 却稳步上升。KIAN 的学习曲线方差也更小；在连续控制上，忽略了熵失衡的 A2T 基本崩溃，KIAN 却保持稳定。

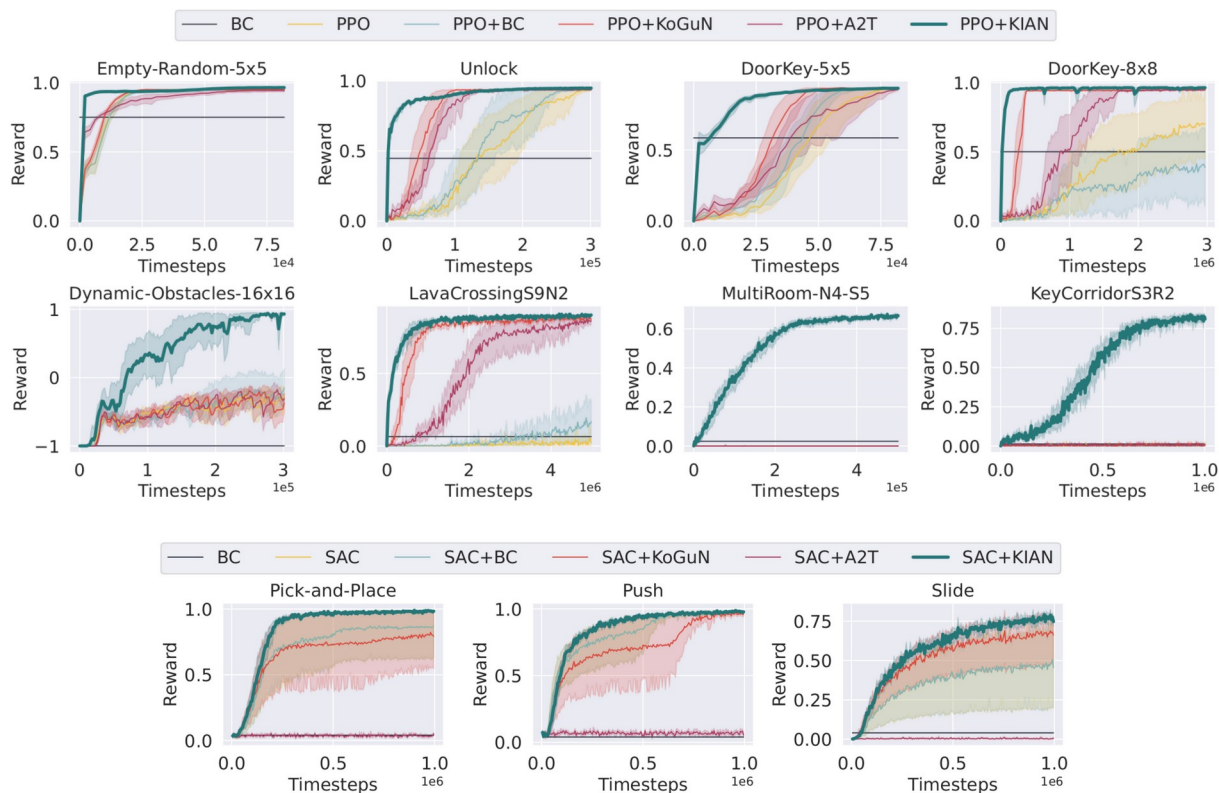


图 3：MiniGrid（上两行）与 OpenAI-Robotics（最后一行）的样本效率学习曲线。在一套单独无法完成任务的知识集下（见贴地的 BC 线），KIAN（加粗青色）用少得多的样本就在所有任务上达到高回报，且在更难的环境上领先幅度更大。

迁移：把知识带到新任务

由于 KIAN 的知识集是模块化的，训练好的智能体可以零样本地投放到不同任务上。在「简单到复杂」（S2C，在小环境上训练、到更大环境上测试）和「复杂到简单」（C2S）的迁移中，KIAN 在多数迁移上都优于所有基线，且方差明显更小。几组代表性数字可以让差距更直观。

表 1：零样本迁移回报（越高越好）：KIAN 与各设定下最强基线的对比。

零样本迁移设定	KIAN	最强基线
Empty-Random 5x5 → 16x16 (S2C)	0.93	0.53 (KoGuN)
Pick-and-Place, 10 倍目标范围 (S2C)	0.72	0.30 (RL+BC)
DoorKey-8x8 → Reach (C2S)	1.00	0.80 (RL / RL+BC)

消融：熵失衡修复是否值得

熵失衡修复到底有多重要？消融实验对比了带与不带修改后策略分布的 KIAN。去掉修改后，在最吃力的任务上性能显著下降，包括 Dynamic-Obstacles-16x16、MultiRoom-N4-S5，以及连续控制的 Pick-and-Place、Push、Slide：一个最大化熵的智能体会坍缩到单一策略上，不再有效探索。打开修改后，高效探索得以恢复，性能也随之回升。这说明该修复是必要的，而非可有可无的点缀。

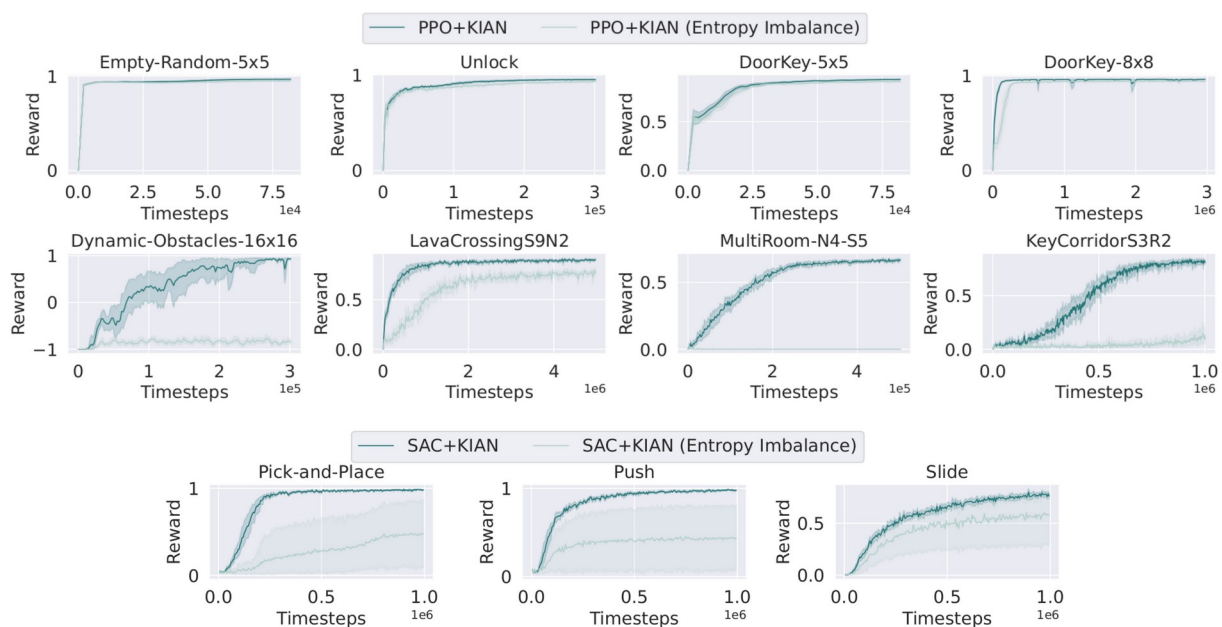


图 5：带与不带熵失衡修复的 KIAN。简单任务上差距很小，难任务上差距很大（Dynamic-Obstacles-16x16、MultiRoom-N4-S5 以及机械臂任务）：修复恢复了高效探索，未修复的版本则停滞不前。

最后，专门的组合与增量实验直接印证了模块化设计的收益：KIAN 复用此前学到的 key 和内部 actor，用比分别从零训练更少的总样本，依次学会两个任务。这正是 Joy 先把摩托车加入技能集、再去学开车这一比喻的可度量版本。

小结

留下的核心思想很简单：把每个知识策略当作一个独立、可由注意力寻址的 key，外部知识便从固定、纠缠的模块变成一组可拼装的积木。智能体可以随时增添、删除或重排自己的策略而无需重新训练；熵失衡修复则保证在融合大量策略时探索依旧高效。最终得到的 RL actor，学得更快、泛化更好，并且始终保持灵活。

同样需要如实看待其边界：这些收益是在网格世界与仿真机械臂基准上、以简单程序化策略构成的知识集验证的；方法如何扩展到更丰富、更高维的知识来源以及真实机器人场景，仍留待未来工作。但作为让智能体像人一样复用与重组技能的一步，KIAN 基于 key 的模块化是一条干净而实用的路径。